

게임 프로그래밍 패턴 — 기말과제

디자인 패턴을 적용한 리팩토링 결과 보고서

Unity 3D 슈팅 프로젝트(2개 모듈)에
State · Object Pool · Observer · Strategy 패턴 적용

이름: _____ 학번: _____

제출일: 2026년 6월 ____일

* 표지의 이름/학번/제출일은 제출 전 직접 기입하세요.

목차

1. 서론
2. 대상 프로젝트 분석 (Before)
3. 디자인 패턴 적용 상세
 - 3.1 State 패턴 — 적 AI 유한상태기계
 - 3.2 Object Pool 패턴 — 제네릭 오브젝트 풀
 - 3.3 Observer 패턴 — 게임 이벤트 버스
 - 3.4 Strategy 패턴 — IDamageable 피해 처리
4. 적용 결과에 대한 종합 의견
5. 결론
6. 부록 A. 변경 파일 목록
7. 부록 B. 컴파일 검증

1. 서론

1.1 과제 목표

본 보고서는 "게임 프로그래밍 패턴" 기말과제로서, 직접 작업한 프로젝트에 **3개 이상의 디자인 패턴**을 적용한 리팩토링 과정과 그 결과를 기록한다. 단순히 패턴을 끼워 맞추는 것이 아니라, **실제 코드에 존재하던 결함도·중복·버그**를 근거로 패턴을 선택하고, 적용 후의 장점뿐 아니라 **부정적 결과와 트레이드오프**까지 비판적으로 검토하는 것을 목표로 한다.

1.2 대상 프로젝트 선정

대상은 Unity 2022.3.20f1로 제작한 3D 슈팅 게임 프로젝트이다. 코드를 분석한 결과, 이 저장소는 사실상 **서로 다른 두 개의 게임 모듈**이 하나의 Unity 프로젝트에 합쳐진 형태였다.

- **모듈 A — 탑다운 슈터(초기 버전):** Enemy, Enemy_Controller, Enemy_Spawner, Bullet, Enemy_Bullet 등. Update()에서 직접 이동·사격하고 Instantiate/Destroy를 그대로 사용하는 단순 구조.
- **모듈 B — 3D FPS(본편):** EnemyFSM(NavMesh 기반 유한상태기계), 무기 시스템(WeaponBase 계층), MemoryPool 오브젝트 풀, PlayerHUD, 폭발 드럼통·아이템 등 비교적 복잡한 구조.

두 모듈은 코드 베이스·설계 수준·해결하는 문제가 뚜렷이 다르므로, 과제에서 요구하는 **"2개 이상의 프로젝트"**를 이 두 모듈로 간주하여 패턴을 적용하였다. 일부 패턴(Observer, Strategy)은 두 모듈 모두에 적용하여, 동일한 패턴이 서로 다른 설계 맥락에서 어떻게 작동하는지도 함께 비교하였다.

1.3 적용한 패턴 개요

패턴	적용 모듈	한 줄 요약
State	B	enum + 문자열 코루틴 FSM을 상태 객체로 분리
Object Pool	B	O(n) 선형탐색 풀을 제네릭 O(1) 풀로 재구현
Observer	A + B	적 사망→점수 결함을 이벤트 버스로 분리
Strategy	A + B	타입 분기 피해 처리를 IDamageable 다형성으로

네 패턴 모두 GoF의 대표 패턴이며, 단순 게터/세터 수준이 아니라 **제어 흐름·객체 생성·객체 간 통신·다형성**이라는 서로 다른 축의 문제를 다룬다.

1.4 검증 방법

리팩토링된 코드는 Unity 에디터 배치모드로 컴파일을 검증하였다(부록 B). 다만 헤드리스 환경에서 플레이모드 런타임 검증은 수행하지 못했다. 이 한계는 4장에서 다시 논의한다.

2. 대상 프로젝트 분석 (Before)

2.1 모듈 B: 3D FPS의 구조와 문제점

모듈 B는 기능이 풍부한 만큼 다음과 같은 구조적 문제를 안고 있었다.

- **문자열 기반 유한상태기계.** EnemyFSM은 적의 상태를 enum으로 두고, 상태 전환을 `StartCoroutine(enemyState.ToString())`처럼 **enum 이름 문자열로** 코루틴을 호출하였다. 상태 이름과 코루틴 메서드 이름이 문자열로 묶여 있어, 오타나 이름 변경 시 컴파일 오류 없이 조용히 동작이 멈춘다.
- **다중 인스턴스에 잘못 쓰인 싱글톤.** EnemyFSM은 적이 여러 마리임에도 `public static EnemyFSM instance`를 두고 Awake에서 자기 자신을 대입한다. 결과적으로 instance는 **마지막에 스폰된 적**을 가리키며, 이를 참조하는 점수 계산이 오작동했다(2.3 참조).
- **O(n) 선형탐색 오브젝트 풀.** MemoryPool은 활성화/비활성화 시 전체 리스트를 매번 순회한다.
- **타입 분기 피해 처리.** 무기는 레이캐스트로 맞은 대상의 태그를 검사해 `GetComponent<■■■■>().TakeDamage()`를 호출하는 if-else 사슬을 갖는다. 피해를 입는 타입이 늘어날 때마다 모든 무기 코드를 수정해야 한다.

2.2 모듈 A: 탑다운 슈터의 구조와 문제점

모듈 A는 더 단순하지만 그만큼 더 거친 문제를 갖는다.

- **싱글톤을 통한 피해 전달.** 플레이어 총알 Bullet은 적을 맞으면 `Enemy.instance.enemyHealth`를 깎는다. 적이 여러 마리여도 항상 instance(마지막 적)의 체력만 깎이는 **명백한 버그**다.
- **직접 호출 결합.** 적 Enemy는 죽을 때 `Score_Manager.instance.IncreaseScore()`를 직접 호출하여 점수 UI와 강하게 결합되어 있다.
- **풀링 부재.** 적 총알을 Instantiate로 생성하고 화면을 벗어나면 Destroy한다.

2.3 발견된 핵심 버그 목록

리팩토링 과정에서 다음과 같은 **실제 버그**를 확인하였다. 이들은 4장의 "패턴이 버그를 어떻게 제거했는가" 논의의 근거가 된다.

#	위치	증상	원인
B1	Score_Manager / EnemyFSM	어떤 적을 죽여도 항상 마지막 스폰된 적의 점수 가 가산됨	EnemyFSM.instance 싱글톤 오용
B2	Bullet / Enemy	한 적을 맞혀도 영동한 적의 체력이 깎임	Enemy.instance 싱글톤 오용
B3	MemoryPool.DeactivateAllPoolItem	활성 객체가 앞쪽에 모여있지 않으면 잘못된 객체 를 비활성화	for(i<activeCount) list[i] 인덱싱 오류
B4	EnemyFSM.Idle	상태를 떠난 뒤에도 자동 전환 코루틴이 살아남아 영동한 시점에 Wander로 전환	ChangeState가 보조 코루틴을 멈추지 않음
B5	Player_Controller, GameManger	런타임 스크립트에 using UnityEditor 포함 → 플레이어 빌드 시 컴파일 실패 위험	에디터 전용 네임스페이스 오용

3. 디자인 패턴 적용 상세

각 절은 "동기 → Before → After → 효과 → 비평" 순서로 기술한다.

3.1 State 패턴 — 적 AI 유한상태기계

동기

EnemyFSM의 상태 전환은 문자열에 의존했고, 각 상태의 행동(Idle/Wander/Pursuit/Attack)이 하나의 거대한 클래스 안에 코루틴으로 뒤섞여 있었다. 상태가 늘어날수록 클래스가 비대해지고, 상태별 진입/종료 처리가 분산되어 추적이 어렵다. 이는 GoF **State 패턴**의 전형적인 적용 대상이다.

Before

```
public enum EnemyState { None=-1, Idle=0, Wander, Pursuit, Attack }

public static EnemyFSM instance;           // (1) misused singleton

public void ChangeState(EnemyState newState)
{
    if (enemyState == newState) return;
    StopCoroutine(enemyState.ToString()); // (2) stop coroutine BY STRING
    enemyState = newState;
    StartCoroutine(enemyState.ToString()); // (3) start coroutine BY STRING
}

private IEnumerator Idle()
{
    StartCoroutine("AutoChangeFromIdleToWander"); // (4) never stopped -> leak
    while (true)
    {
        CalculateDistanceToTargetAndSelectState();
        yield return null;
    }
}
```

(2)(3)은 enum 이름과 메서드 이름이 문자열로 묶여 취약하고, (4)의 보조 코루틴은 상태를 떠나도 멈추지 않아 버그 B4를 유발한다.

After

상태를 인터페이스로 추상화하고, 상태별 클래스로 분리하였다. EnemyFSM은 **컨텍스트(Context)** 역할만 맡는다.

```
public interface IEnemyState
{
    EnemyState StateType { get; }
    void Enter(); // once when entering
    void Execute(); // every frame
    void Exit(); // once when leaving
}

// EnemyFSM (context): owns the active state and switches between them
private readonly Dictionary<EnemyState, IEnemyState> states = new();
private IEnemyState currentState;

private void Update() => currentState?.Execute();

public void ChangeState(EnemyState newState)
{
    if (currentStateType == newState) return;
    currentState?.Exit();
```

```

        currentStateType = newState;
        currentState = states[newState];    // type-safe lookup, no strings
        currentState.Enter();
    }

    // One concrete state. Its own timer lives here, so it cannot leak (fixes B4)
    public class EnemyIdleState : IEnemyState
    {
        private readonly EnemyFSM enemy;
        private float idleTimer, timeToWander;

        public void Enter() { idleTimer = 0f; timeToWander = Random.Range(1f, 5f); }

        public void Execute()
        {
            enemy.CalculateDistanceToTargetAndSelectState();
            if (enemy.CurrentState != EnemyState.Idle) return;
            idleTimer += Time.deltaTime;
            if (idleTimer >= timeToWander) enemy.ChangeState(EnemyState.Wander);
        }

        public void Exit() { }
    }

```

구조 변화는 다음과 같다.

[Before]	[After]
EnemyFSM	EnemyFSM (Context)
+ enum state	- Dictionary<EnemyState, IEnemyState>
+ Idle() coroutine	- currentState.Execute()
+ Wander() coroutine	
+ Pursuit() coroutine	IEnemyState (interface)
+ Attack() coroutine	/ \
+ string transitions	Idle Wander Pursuit Attack (classes)

효과

- 문자열 전환이 사라지고 Dictionary 조회로 바뀌어 **오타-이름변경에 의한 무음 실패가 불가능**해졌다.
- 상태별 진입/종료/매 프레임 행동이 한 클래스에 응집되어 가독성이 향상되었다.
- 코루틴 누수(B4)와 싱글톤(B1의 한 축)이 제거되었다.

비평 (트레이드오프)

State 패턴이 항상 이득인 것은 아니다.

- 클래스 수 증가.** 4개 상태를 위해 인터페이스 1 + 상태 4 + 컨텍스트 변경까지 **파일이 6개로** 늘었다. 상태가 4개뿐이고 전환 규칙이 단순한 이 FSM에서는, 코루틴 한 파일이 더 간결하다고 볼 여지도 있다. 패턴의 이득은 상태/전환이 더 많아질수록 커진다.
- 컨텍스트와 상태의 결합.** 상태가 enemy.Agent, enemy.Status 등 컨텍스트 내부에 접근해야 해서, EnemyFSM에 internal 접근자를 다수 노출했다. 캡슐화가 다소 약해졌으며, 더 엄격히 하려면 상태에 필요한 인터페이스를 별도로 정의해야 한다.
- 디버깅 흐름의 분산.** 하나의 코루틴을 위에서 아래로 읽던 방식과 달리, 행동이 여러 파일에 흩어져 전체 흐름 파악에 파일 간 이동이 필요해졌다.

3.2 Object Pool 패턴 — 제네릭 오브젝트 풀

동기

MemoryPool은 이미 Object Pool 패턴을 쓰고 있었지만, (1) 활성/비활성화가 O(n) 선형탐색이고, (2) DeactivateAllPoolItem에 인덱싱 버그(B3)가 있으며, (3) 풀마다 GetComponent를 반복 호출하는 래퍼가 중복되어 있었다. 즉 **패턴은 있으나 구현이 미흡했다**. 이를 올바르게 재구현하고 **제네릭화**하였다.

Before

```
public GameObject ActivatePoolItem()
{
    if (maxCount == activeCount) InstantiateObjects();
    for (int i = 0; i < poolItemList.Count; ++i) // O(n) scan
    {
        if (poolItemList[i].isActive == false) { /* activate, return */ }
    }
    return null;
}

public void DeactivateAllPoolItem()
{
    for (int i = 0; i < activeCount; ++i) // BUG (B3):
    {
        PoolItem poolItem = poolItemList[i]; // indexes [0,activeCount)
        // ... deactivates the WRONG items if actives aren't packed at the front
    }
}
```

After

자유 목록은 Queue, 객체→항목 매핑은 Dictionary로 두어 활성/비활성화를 **O(1)**로 만들고, 전체 비활성화는 실제 활성 항목만 순회하도록 고쳐 B3를 제거했다.

```
private readonly Queue<PoolItem> inactiveItems = new(); // O(1) activate
private readonly Dictionary<GameObject, PoolItem> lookup = new(); // O(1) release

public GameObject ActivatePoolItem()
{
    if (inactiveItems.Count == 0) InstantiateObjects();
    PoolItem item = inactiveItems.Dequeue();
    item.isActive = true; item.gameObject.SetActive(true);
    activeCount++;
    return item.gameObject;
}

public void DeactivatePoolItem(GameObject obj)
{
    if (!lookup.TryGetValue(obj, out var item) || !item.isActive) return;
    item.isActive = false; item.gameObject.SetActive(false);
    inactiveItems.Enqueue(item); activeCount--;
}
```

그 위에 **제네릭 래퍼** ObjectPool<T>를 도입하였다. 컴포넌트를 직접 돌려주고 캐시하여, 호출부의 반복적인 GetComponent<T>()를 제거한다.

```
public class ObjectPool<T> where T : Component
{
    private readonly MemoryPool pool;
    private readonly Dictionary<GameObject, T> components = new();

    public T Get()
    {
        GameObject go = pool.ActivatePoolItem();
        if (!components.TryGetValue(go, out T c)) { c = go.GetComponent<T>(); components[go] = c; }
        return c; // GetComponent paid once per object, not per spawn
    }

    public void Release(T c) => pool.DeactivatePoolItem(c.gameObject);
}
```

```
}
```

탄피(Casing)·탄흔(Impact) 서브시스템을 이 제네릭 풀로 이전하였다. 호출부는 다음과 같이 단순해진다.

```
// Before: GameObject item = memoryPool.ActivatePoolItem();  
//         item.GetComponent<Casing>().Setup(memoryPool, dir);  
// After :  
Casing casing = pool.Get();  
casing.Setup(pool, direction);
```

효과

- 스폰이 잦은 탄피·탄흔·총알에서 활성/비활성화가 $O(n) \rightarrow O(1)$ 로 개선되어, 풀이 커질수록 프레임 비용이 안정적이다.
- 전체 비활성화 버그(B3)가 제거되었다.
- 타입 안전성이 생기고 호출부에서 GetComponent 반복이 사라졌다.

비평 (트레이드오프)

- **Unity 직렬화와 제네릭의 불협화음.** 제네릭 MonoBehaviour는 인스펙터에 직렬화되지 않으므로, 프리팹을 받는 CasingMemoryPool 같은 **얇은 래퍼 컴포넌트는 여전히 필요하다.** 즉 제네릭화가 래퍼를 완전히 없애주지는 못했다.
- **풀의 단위가 프리팹이라는 제약.** MemoryPool/ObjectPool<T> 인스턴스 하나는 **프리팹 하나만** 관리한다(ObjectPool<T>의 타입 인자도 "여러 프리팹 수용"이 아니라 "꺼낼 때의 타입 안전"을 뜻한다). 따라서 탄흔처럼 프리팹이 여러 종류인 곳은 호출부가 풀 배열을 직접 구성하고 enum↔배열 인덱스를 수동으로 맞춰야 하며(ImpactMemoryPool), 종류가 늘수록 관리 부담과 실수 여지가 커진다. 프리팹을 키로 풀을 자동 생성·조회하는 풀 레지스트리(pool of pools)로 개선할 수 있으나, "어떤 프리팹이 풀링되는지"가 코드에서 드러나지 않게 되는 새로운 비용이 생겨 이번 범위에서는 도입하지 않았다.
- **메모리 상주 증가.** Object Pool 패턴 자체의 본질적 트레이드오프로, 풀은 사용하지 않는 객체도 메모리에 계속 들고 있다. 스폰이 드문 객체에는 오히려 낭비다.
- **부분 적용의 비밀관성.** 위험을 줄이기 위해 EnemyMemoryPool은 제네릭으로 이전하지 않고 개선된 MemoryPool을 그대로 쓰게 두었다. 결과적으로 두 스타일이 공존한다. 일관성과 리스크 사이의 의도적 타협이다.

3.3 Observer 패턴 — 게임 이벤트 버스

동기

버그 B1의 근본 원인은 **점수 시스템과 적이 전역 상태로 강하게 결합된** 것이었다. 적이 죽으면 점수가 올라야 한다 — 이 "사건과 반응"의 관계는 GoF **Observer 패턴**으로 분리하는 것이 자연스럽다.

Before

```
// Score_Manager  
public void IncreaseScore()  
{  
    totalScore += EnemyFSM.instance.enemyScore; // (B1) always the LAST enemy  
    UpdateScoreUI();  
}  
  
// EnemyFSM.TakeDamage (on death)  
Score_Manager.instance.IncreaseScore(); // tight coupling
```


적은 Score_Manager를 알고, Score_Manager는 다시 EnemyFSM의 전역 인스턴스를 들여다본다. 양방향으로 얹혀 있고, 점수 값마저 틀린다.

After

발행/구독을 중계하는 가벼운 정적 이벤트 버스를 도입했다. 죽는 적이 자신의 점수를 발행하고, 점수 UI는 단지 구독만 한다.

```

public static class GameEvents
{
    public static event Action<int> EnemyKilled;           // arg = score
    public static void RaiseEnemyKilled(int score) => EnemyKilled?.Invoke(score);
}

// EnemyFSM.TakeDamage (on death): publish OWN score (fixes B1)
GameEvents.RaiseEnemyKilled(enemyScore);

// Score_Manager: pure subscriber, knows nothing about enemies
private void OnEnable() => GameEvents.EnemyKilled += OnEnemyKilled;
private void OnDisable() => GameEvents.EnemyKilled -= OnEnemyKilled;
private void OnEnemyKilled(int score) { totalScore += score; UpdateScoreUI(); }

[Before]  Enemy ■■calls■■> Score_Manager ■■reads■■> EnemyFSM.instance
          (■■■ ■■■ + ■■ ■■)

[After]   Enemy ■■raise(score)■■> GameEvents ■■notify■■> Score_Manager
          ■■■■■■■■■> (■■ ■■■ ■■■■ ■■)

```

동일한 패턴을 모듈 A의 Enemy에도 적용하여, 죽을 때 `GameEvents.RaiseEnemyKilled(enemyScore)`를 발행하도록 통일했다.

효과

- 적과 점수 UI의 결합이 끊겨, 적은 누가 점수를 쓰는지 몰라도 된다.
- 각 적이 자기 점수를 발행하므로 B1이 제거되었다.
- 사운드:업적:콤보 등 새로운 구독자를 적 코드 수정 없이 추가할 수 있다.
- 모듈 A와 B가 같은 이벤트 채널을 공유한다.

비평 (트레이드오프)

- **정적 이벤트의 수명 관리 위험.** static event는 씬을 다시 로드해도 살아남는다. 구독 해제(OnDisable)를 누락하면 **중복 호출-메모리 누수**가 발생한다. 즉 Observer는 B1을 고치는 대신 **새로운 함정**을 들여온다. 본 구현은 OnEnable/OnDisable 쌍으로 방어했다.
- **제어 흐름 추적 난이도.** "이 이벤트를 누가 듣는가?"가 코드에 드러나지 않아, 직접 호출보다 흐름을 따라가기 어렵다. 디버깅 시 호출 스택만으로는 인과를 파악하기 힘들다.
- **대안과의 비교.** 더 Unity다운 방법은 ScriptableObject 기반 이벤트 채널이다. 에셋으로 채널을 만들면 인스펙터에서 연결을 관리할 수 있으나, 에셋-연결 설정이라는 비용이 든다. 본 과제에서는 씬 의존을 만들지 않으려고 정적 이벤트를 택했다.

3.4 Strategy 패턴 — IDamageable 피해 처리

동기

무기·폭발·총알이 피해를 줄 때마다 CompareTag + GetComponent<■■■■>()로 분기했다(B2의 근원이기도 하다). "피해를 받을 수 있다"는 **행동을 인터페이스로 추상화**하면, 호출부는 구체 타입을 몰라도 된다. 이는 Strategy 패턴(행동의 다형적 캡슐화)의 적용이다.

Before

```
// WeaponPistol.TwoStepRaycast
if (hit.transform.CompareTag("Enemy"))
    hit.transform.GetComponent<EnemyFSM>().TakeDamage(damage);
else if (hit.transform.CompareTag("InteractionObject"))
    hit.transform.GetComponent<InteractionObject>().TakeDamage(damage);

// Bullet (Module A): wrong target via singleton (B2)
Enemy.instance.enemyHealth -= bullet_Damage;
```

After

```
public interface IDamageable { void TakeDamage(int damage); }
```

EnemyFSM, Player_Controller, InteractionObject(및 그 하위 배럴/타깃), 모듈 A의 Enemy가 모두 이 인터페이스를 구현한다(대부분 이미 TakeDamage(int)를 갖고 있어 인터페이스 선언만 추가). 호출부는 다음 한 줄로 통일된다.

```
// WeaponPistol / WeaponAssault
hit.transform.GetComponent<IDamageable>()?.TakeDamage(weaponSetting.damage);

// Bullet (Module A): hits whatever it actually collided with (fixes B2)
other.GetComponent<IDamageable>()?.TakeDamage(bullet_Damage);
```

효과

- 태그-타입 분기 사슬이 사라지고, 새로운 피해 대상 추가 시 **무기 코드를 건드리지 않는다**(개방-폐쇄 원칙).
- 모듈 A의 싱글톤 관통 버그(B2)가 자연스럽게 제거되었다.
- 모듈 A-B의 전투 코드가 동일한 추상화를 공유한다.

비평 (트레이드오프)

- **단순 인터페이스로 표현하기 어려운 정책.** ExplosionBarrel은 대상 종류별로 다른 피해량을 준다(플레이어 50, 적 300, 상호작용 오브젝트 100). 단일 TakeDamage(int)로는 이 "공격자가 대상별로 다르게 대한다"는 규칙을 담기 어렵다. 더블 디스패치(Visitor)나 대상이 자신의 취약도를 갖는 설계가 필요하다. 그래서 본 리팩토링에서 ExplosionBarrel은 의도적으로 그대로 두었다 — Strategy/인터페이스 다형성의 한계를 보여주는 사례다.
- **태그 검사의 잔존.** 탄흔 효과 종류는 여전히 태그로 분기한다. 이는 "피해"가 아니라 "표현"의 문제라 IDamageable의 범위 밖이며, 무리하게 통합하지 않았다.
- **널 처리의 산재.** GetComponent<IDamageable>()?.의 널 조건 연산이 호출부마다 반복된다.

4. 적용 결과에 대한 종합 의견

4.1 정량적 변화

항목	Before	After
적 AI 상태 관리	enum + 문자열 코루틴(1파일)	상태 객체 6파일(인터페이스+4상태+컨텍스트)
풀 활성/비활성화 복잡도	O(n)	O(1)
적 사망 → 점수 결합	양방향 직접 참조	단방향 이벤트
피해 처리 분기	태그/타입 if-else	IDamageable 단일 호출
확인된 버그	B1~B5 존재	B1~B5 제거
신규/수정 파일	—	신규 8, 수정 15

4.2 긍정적 결과

가장 큰 수확은 **패턴 적용이 실제 버그 제거로 직결**되었다는 점이다. State는 코루틴 누수(B4)를, Observer/Strategy는 싱글톤 오용 버그(B1·B2)를, Object Pool 재구현은 인덱싱 버그(B3)를 없앴다. "좋은 구조"가 "정확한 동작"으로 이어진 셈이다. 또한 모듈 A·B에 같은 패턴(Observer-Strategy)을 적용하면서, **결합도 감소**와 **확장 지점의 일원화**라는 이득을 두 맥락에서 일관되게 확인했다.

4.3 부정적 결과 및 트레이드오프 (핵심)

과제 요구대로, **좋은 결과만 있었던 것은 아니다**.

- 과설계의 위험.** State 패턴은 4개 상태짜리 FSM에 파일 6개를 요구했다. 규모가 작을 때는 패턴이 오히려 진입장벽과 탐색비용을 늘린다. "패턴을 위한 패턴"이 되지 않도록, 상태/전환의 복잡도가 임계점을 넘을 때 도입하는 것이 옳다.
- 새로운 함정의 유입.** Observer의 정적 이벤트는 B1을 고치는 대가로 **구독 해제 누락 시 누수**라는 위험을 새로 들였다. 패턴은 문제를 "다른 종류의 문제"로 바꾸는 것이지, 공짜가 아니다.
- 추상화가 못 담은 정책.** IDamageable은 대상별 차등 피해(ExplosionBarrel)를 담지 못해 적용을 보류했다. 모든 곳에 같은 추상화를 강요하면 오히려 왜곡이 생긴다.
- 프레임워크와의 마찰.** 제네릭 ObjectPool<T>는 Unity의 직렬화 한계로 래퍼를 완전히 제거하지 못했다. 패턴은 진공이 아니라 **플랫폼 제약 위에서** 타협된다.
- 검증의 한계.** 본 작업은 배치모드 **컴파일 검증**까지만 수행했고, 플레이모드에서의 실제 동작(상태 전환 타이밍, 풀 재사용 시각 효과 등)은 검증하지 못했다. 구조적 등가성은 확인했으나, 런타임 동등성은 추후 플레이 테스트로 보강해야 한다.

4.4 적용을 검토했으나 보류한 패턴

좋은 리팩토링은 "무엇을 적용했는가"만큼 "무엇을 적용하지 않았는가"도 중요하다. 다음 두 패턴은 검토했으나 이번 범위에서 의도적으로 제외했다.

- Command 패턴 (입력 처리).** Player_Controller는 Input.GetKeyDown(...)을 Update에 하드코딩한다. 입력을 커맨드 객체로 캡슐화하면 키 리매핑·입력 리플레이·매크로가 쉬워진다. 그러나 현재 게임에는 키 변경 기능 자체가 없어, 지금 도입하면 **YAGNI(필요해지기 전엔 만들지 말**

것) 원칙에 어긋나는 과설계가 된다. 요구가 실제로 생길 때 도입하는 편이 낫다고 판단했다.

- **Factory 패턴 (생성 분리).** 적·아이템·이펙트의 생성 로직이 여러 곳에 흩어져 있어 Factory로 모을 수 있다. 다만 이미 Object Pool이 "생성/재사용" 책임의 상당 부분을 맡고 있어, 별도 Factory를 두면 **중복 추상화**가 될 위험이 있었다. Pool과 Factory의 책임 경계를 먼저 정리하는 선행 작업이 필요하다고 보아 보류했다.

요컨대 패턴은 "적용 가능하다"와 "적용해야 한다"가 서로 다르며, **문제와 비용이 맞아떨어질 때만** 도입하는 절제 또한 설계 역량의 일부다.

4.5 싱글톤에 대한 메모

이번 리팩토링에서 가장 큰 교훈은 **싱글톤의 오용**이었다. EnemyFSM·Enemy처럼 본질적으로 다중 인스턴스인 대상에 싱글톤을 붙이면, 컴파일은 통과하지만 런타임에 "마지막 하나"만 살아남아 조용히 오작동한다(B1·B2). 싱글톤은 "전역 접근"이라는 편의를 주지만, 그 편의가 곧 **숨은 결합과 상태 버그**의 통로가 된다. 본 작업에서는 이런 싱글톤을 제거하고, 통신은 Observer로, 다형성은 Strategy로 대체했다. 죽어 있던 Player_Controller.instance도 함께 제거했다.

5. 결론

세 개를 초과하는 네 개의 패턴(State-Object Pool-Observer-Strategy)을 두 모듈에 적용하여, 단순한 구조 개선을 넘어 **실재하던 다섯 개의 버그(B1~B5)** 를 제거했다. 동시에 각 패턴이 가져오는 비용 — 클래스 증가, 이벤트 수명 관리, 추상화의 한계, 프레임워크 마찰, 검증 범위 — 도 구체적으로 확인했다. 패턴은 만능이 아니라 **문제의 성격에 맞을 때** 가치가 있으며, 적용 후에도 새로운 트레이드오프를 계속 관리해야 한다는 것이 본 과제의 결론이다.

향후 작업으로는 (1) EnemyMemoryPool의 제네릭 풀 이전, (2) ExplosionBarrel의 대상별 피해를 위한 더블 디스패치 도입, (3) 정적 이벤트의 ScriptableObject 채널 전환, (4) 플레이모드 런타임 검증을 제안한다.

부록 A. 변경 파일 목록

신규 (8)

- `IEEnemyState.cs`, `EnemyIdleState.cs`, `EnemyWanderState.cs`, `EnemyPursuitState.cs`, `EnemyAttackState.cs` — State 패턴
- `ObjectPool.cs` — 제네릭 오브젝트 풀
- `GameEvents.cs` — Observer 이벤트 버스
- `IDamageable.cs` — Strategy 인터페이스

수정 (15)

- `EnemyFSM.cs` — 컨텍스트화, 싱글톤 제거, 이벤트 발행, `IDamageable`
- `MemoryPool.cs` — $O(1)$ 재구현, B3 수정
- `Casing.cs`, `CasingMemoryPool.cs`, `Impact.cs`, `ImpactMemoryPool.cs` — 제네릭 풀 이전
- `Score_Manager.cs` — 순수 구독자화, 싱글톤 제거
- `Enemy.cs`, `Bullet.cs` — Observer-Strategy 적용, 싱글톤 제거(B2)
- `WeaponPistol.cs`, `WeaponAssault.cs`, `Enemy_Bullet.cs` — `IDamageable` 호출
- `InteractionObject.cs` — `IDamageable` 구현
- `Player_Controller.cs`, `GameManger.cs` — 죽은 싱글톤·using `UnityEditor` 제거(B5)

부록 B. 컴파일 검증

Unity 2022.3.20f1 배치모드로 전체 스크립트를 재컴파일하여 검증하였다.

```
Unity.exe -batchmode -quit -nographics -projectPath <proj> -logFile <log>
```

결과: 로그에 error CS#### 0건, Library/ScriptAssemblies/Assembly-CSharp.dll이 새로 빌드됨(컴파일 성공). 신규 스크립트 8개의 .meta 파일도 정상 생성되었다. (로그의 라이선스 경고는 컴파일과 무관하다.) 단, 플레이모드 런타임 검증은 수행하지 않았다(4.3-5 참조).